Key Insights and Implications

Here are some critical insights into implementing Strangler Fig pattern effectively:

1. **Proxy Layer**: At the heart of the Strangler Fig Pattern is the proxy or router layer. It's like the gatekeeper of a magical castle, directing visitors (requests) to the right chamber (service). This layer needs to be robust and might have logic based on:
    - Feature flags: Turning features on/off to test new functionalities.
    - User segmentation: Routing certain users to new services for A/B testing or phased rollouts.
    - API versioning: Directing requests based on version headers or paths.
2. **Data Consistency**: Often, one of the trickiest parts is managing data between the old and new systems, especially if both systems are writing to the database. You might need:
    - Database replication.
    - Synchronizing data changes between the two systems using events or shared queues.
    - Adopting an "Event-Driven" approach where changes are propagated through events rather than direct database writes.
3. **Monitoring & Feedback**: This isn't just about code—it's about ensuring the new system meets or exceeds the old system's performance, reliability, and usability.
    - Implement comprehensive logging and monitoring.
    - Collect feedback from users affected by the changes.
4. **Parallel Running**: There's a cost to running two systems in parallel. This includes infrastructure costs, complexity, and potential for data inconsistencies. The goal should always be to fully retire the old system as soon as feasible.
5. **Decoupling**: The older system might have tightly coupled components. Before introducing the new system, you may need to do some refactoring to decouple these components, making them easier to replace.
6. **State Management**:
    - As you transition, managing state between old and new systems becomes crucial.
    - **Shared state**: Systems might need to access shared caches or distributed storage to ensure both old and new parts can access and modify a unified state.
    - **State synchronization**: If each system maintains its own state, consider synchronization mechanisms like CRDTs (Conflict-free Replicated Data Types) to merge states without conflicts.
7. **Service Communication**:
    - As new microservices grow, they might need to communicate with each other and the old system.
    - **Event sourcing**: Store the state of business entities as a sequence of events. It allows new and old systems to consume and produce events ensuring data consistency.
    - **Message Brokers**: Systems like RabbitMQ or Kafka can be used to ensure proper communication between new microservices and the legacy system.
8. **Rollback Strategy**:
    - Sometimes things go wrong. Prepare a rollback strategy to revert to the old system seamlessly if needed.
    - Use feature toggles or dynamic routing to easily switch between systems.
9. **Testing**:
    - **Contract Testing**: As you strangle the monolith, ensure that the new microservices adhere to expected behaviors.
    - **End-to-End Testing**: Ensure the whole system (old + new) works harmoniously.
    - **Performance Testing**: The new services should not degrade performance. Load test them to make sure they're up to the mark!
10. **Zero-Downtime Migration**:

- As you're moving functionalities, it's paramount to ensure users experience zero downtimes.
- Techniques like **Blue-Green Deployment** or **Canary Releases** can be used. For instance, roll out the new service to a small percentage of users, observe, and then increase the rollout gradually.

11. **Data Transformation & Migration**:

- If the new system uses a different data model or database, you might need data transformation tools or middleware.
- Tools like Apache **Kafka Connect** can stream data between databases, transforming it on the fly.

12. **Security Considerations**:

- New microservices introduce new potential security vulnerabilities.
- Ensure security practices like API gateway throttling, service-to-service authentication (mTLS), and regular vulnerability scanning.

Organizational & Cultural Aspects:

1. **Cross-functional Teams**: To strangle a monolith effectively, having teams with skills ranging from the legacy system to the new tech stack is beneficial. These teams can work together to ensure smooth transitions.
2. **Feedback Loops**: It's not just about technology. Keep open communication channels with stakeholders, end-users, and operation teams. Their feedback will be invaluable.
3. **Continuous Learning**: As challenges arise, document them, and create knowledge bases. Encourage teams to learn from mistakes and celebrate successes.

← Previous
Strangler Pattern: A Detailed Example

Next →
Introduction To The API Gateway Pattern

☐ Mark as Completed